



Chapter 2: Intro to Relational Model

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use

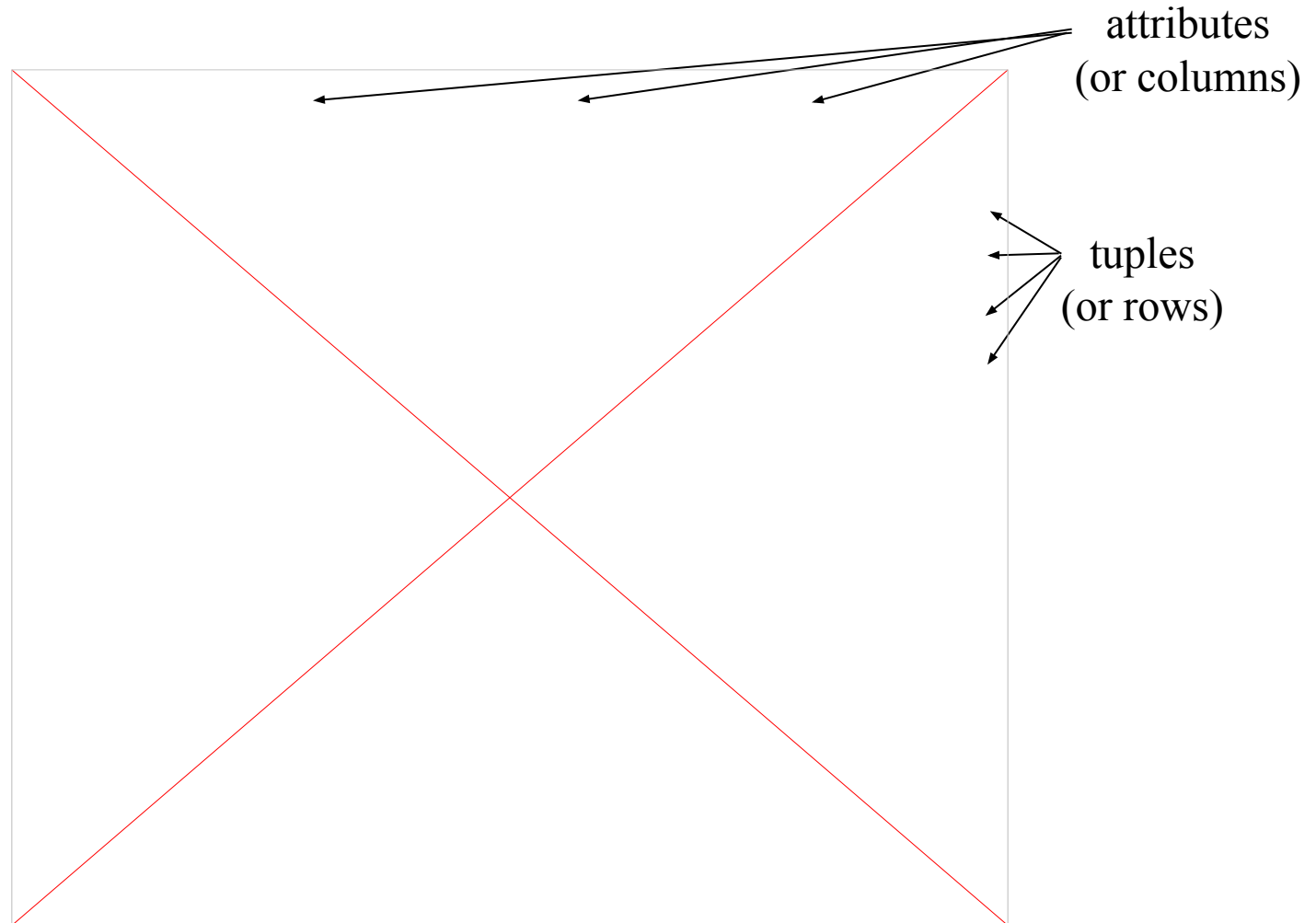


Outline

- Structure of Relational Databases
- Database Schema
- Keys
- Schema Diagrams
- Relational Query Languages
- The Relational Algebra



Example of an *Instructor* Relation





Relation Schema and Instance

- $A_1, A_2, \dots, A_n$ are **attributes**
- $R = (A_1, A_2, \dots, A_n)$ is a **relation schema**

Example:

instructor = (*ID*, *name*, *dept_name*, *salary*)

- A **relation instance** r defined over schema R is denoted by $r(R)$.
- The current values a relation are specified by a table
- An element t of relation r is called a **tuple** and is represented by a **row** in a table



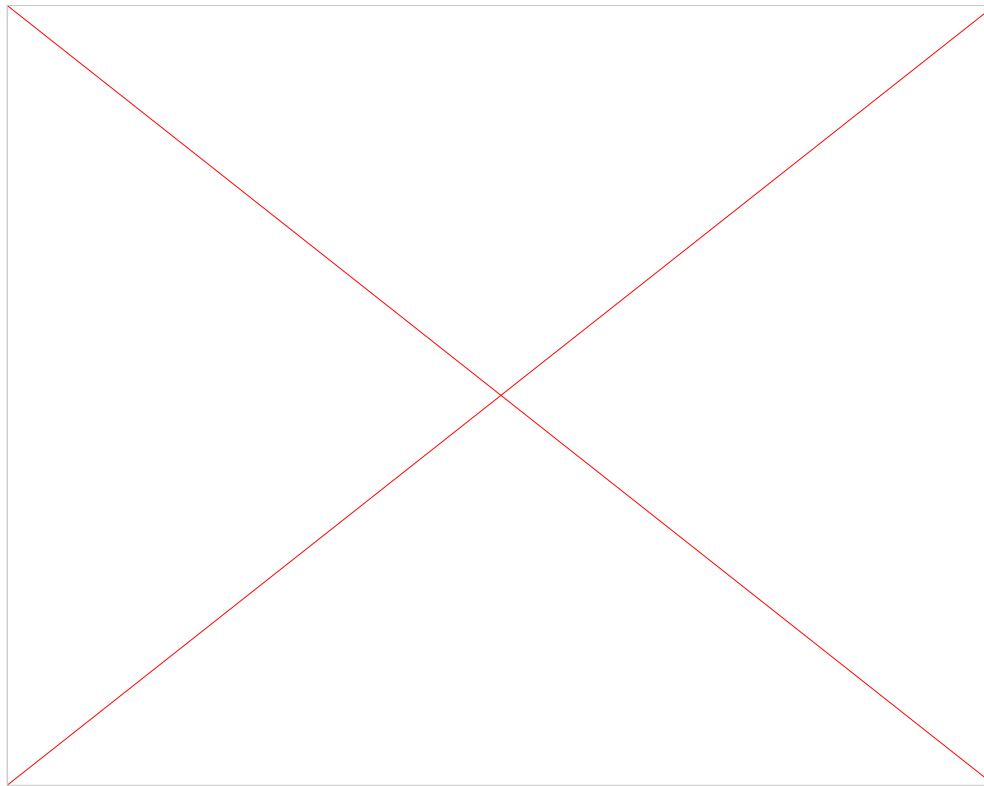
Attributes

- The set of allowed values for each attribute is called the **domain** of the attribute
- Attribute values are (normally) required to be **atomic**; that is, indivisible
- The special value ***null*** is a member of every domain. Indicated that the value is “unknown”
- The null value causes complications in the definition of many operations



Relations are Unordered

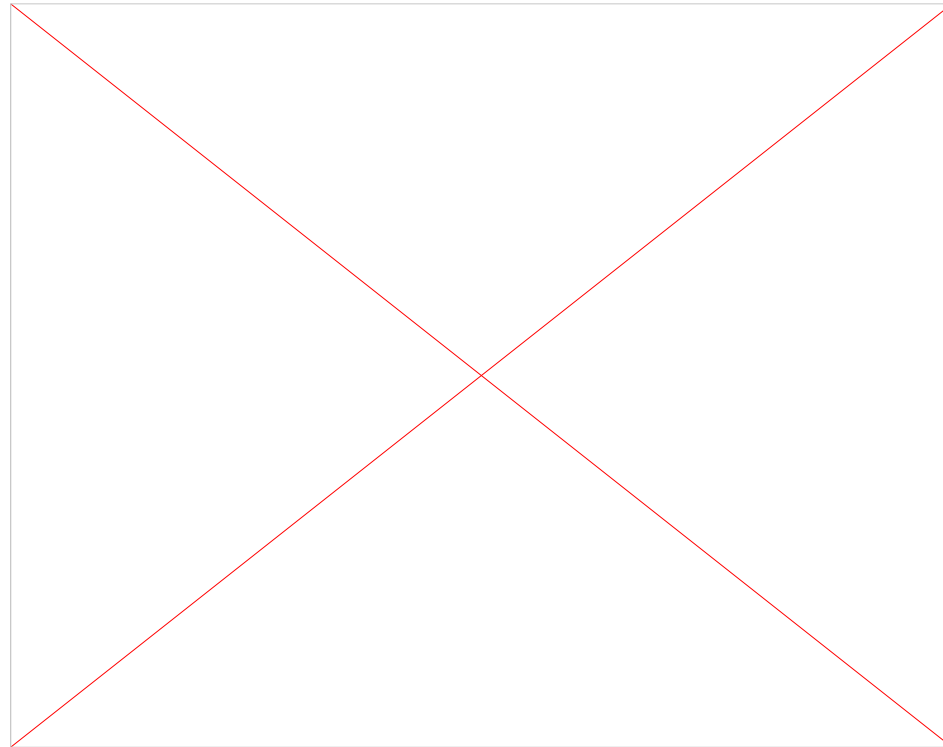
- Order of tuples is irrelevant (tuples may be stored in an arbitrary order)
- Example: *instructor* relation with unordered tuples





Database Schema

- **Database schema** -- is the logical structure of the database.
- **Database instance** -- is a snapshot of the data in the database at a given instant in time.
- Example:
 - schema: *instructor (ID, name, dept_name, salary)*
 - Instance:



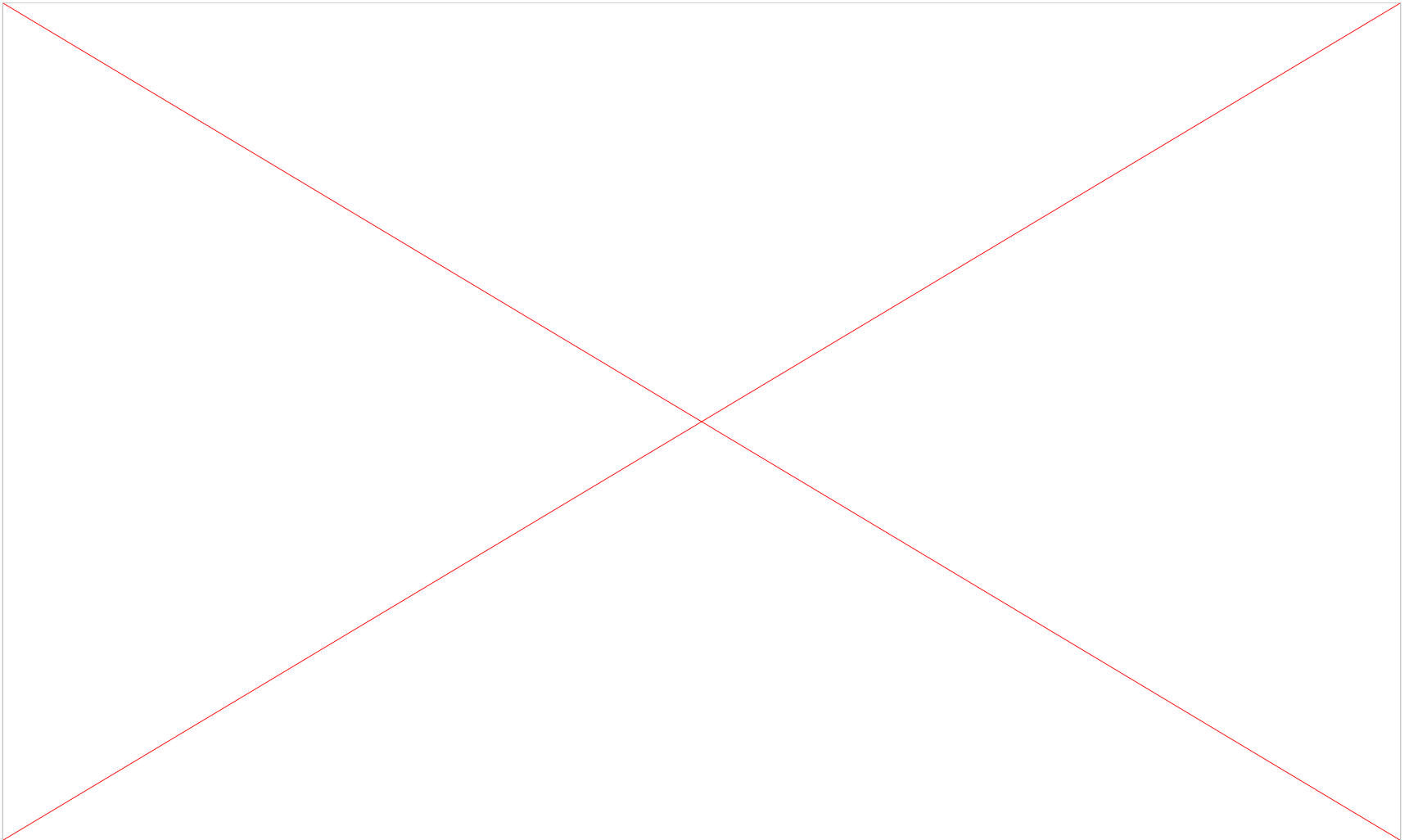


Keys

- Let $K \subseteq R$
- K is a **superkey** of R if values for K are sufficient to identify a unique tuple of each possible relation $r(R)$
 - Example: $\{ID\}$ and $\{ID, name\}$ are both superkeys of *instructor*.
- Superkey K is a **candidate key** if K is minimal
Example: $\{ID\}$ is a candidate key for *Instructor*
- One of the candidate keys is selected to be the **primary key**.
 - Which one?
- **Foreign key** constraint: Value in one relation must appear in another
 - **Referencing** relation
 - **Referenced** relation
 - Example: *dept_name* in *instructor* is a foreign key from *instructor* referencing *department*



Schema Diagram for University Database





Relational Query Languages

- Procedural versus non-procedural, or declarative
- “Pure” languages:
 - Relational algebra
 - Tuple relational calculus
 - Domain relational calculus
- The above 3 pure languages are equivalent in computing power
- We will concentrate in this chapter on relational algebra



Relational Algebra

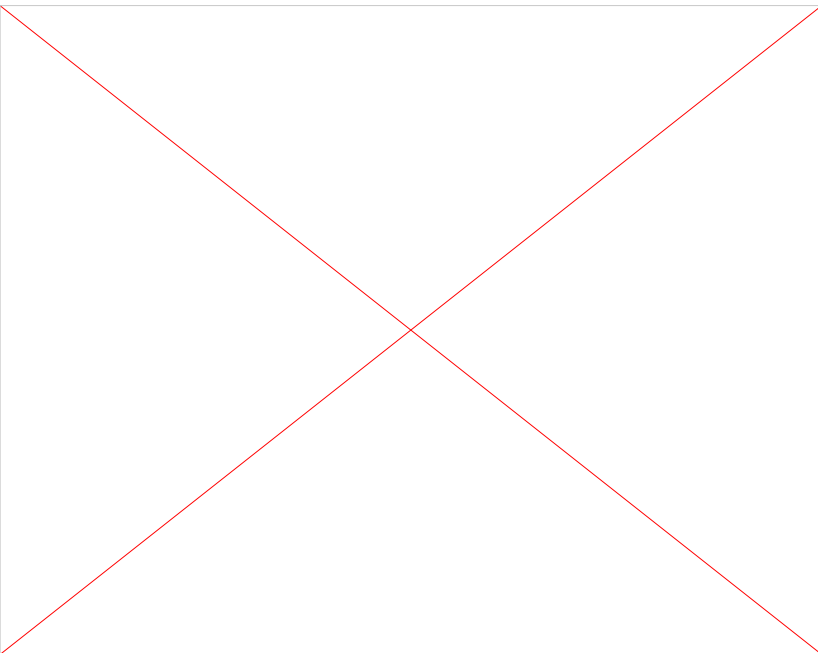
- An algebraic language consisting of a set of operations that take one or two relations as input and produce a new relation as their result.
- Six basic operators
 - select: σ
 - project: Π
 - union: $\cup$
 - set difference: $-$
 - Cartesian product: $\times$
 - rename: ρ



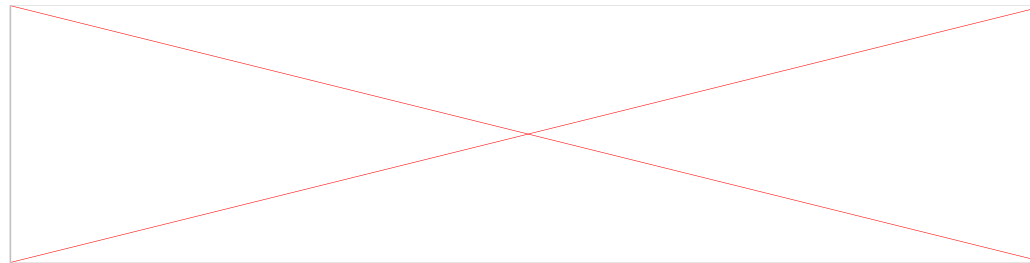
Select Operation

- The **select** operation selects tuples that satisfy a given predicate.
- Notation: $\sigma_p(r)$
- p is called the **selection predicate**
- Example: select those tuples of the *instructor* relation where the instructor is in the “Physics” department.
 - Query

$$\sigma_{dept_name = \text{“Physics”}}(instructor)$$



Result





Select Operation (Cont.)

- We allow comparisons using

$=, \neq, >, \geq, <, \leq$

in the selection predicate.

- We can combine several predicates into a larger predicate by using the connectives:

$\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)

- Example: Find the instructors in Physics with a salary greater \$90,000, we write:

$\sigma_{dept_name="Physics" \wedge salary > 90,000}(instructor)$

- The select predicate may include comparisons between two attributes.
 - Example, find all departments whose name is the same as their building name:
 - $\sigma_{dept_name=building}(department)$

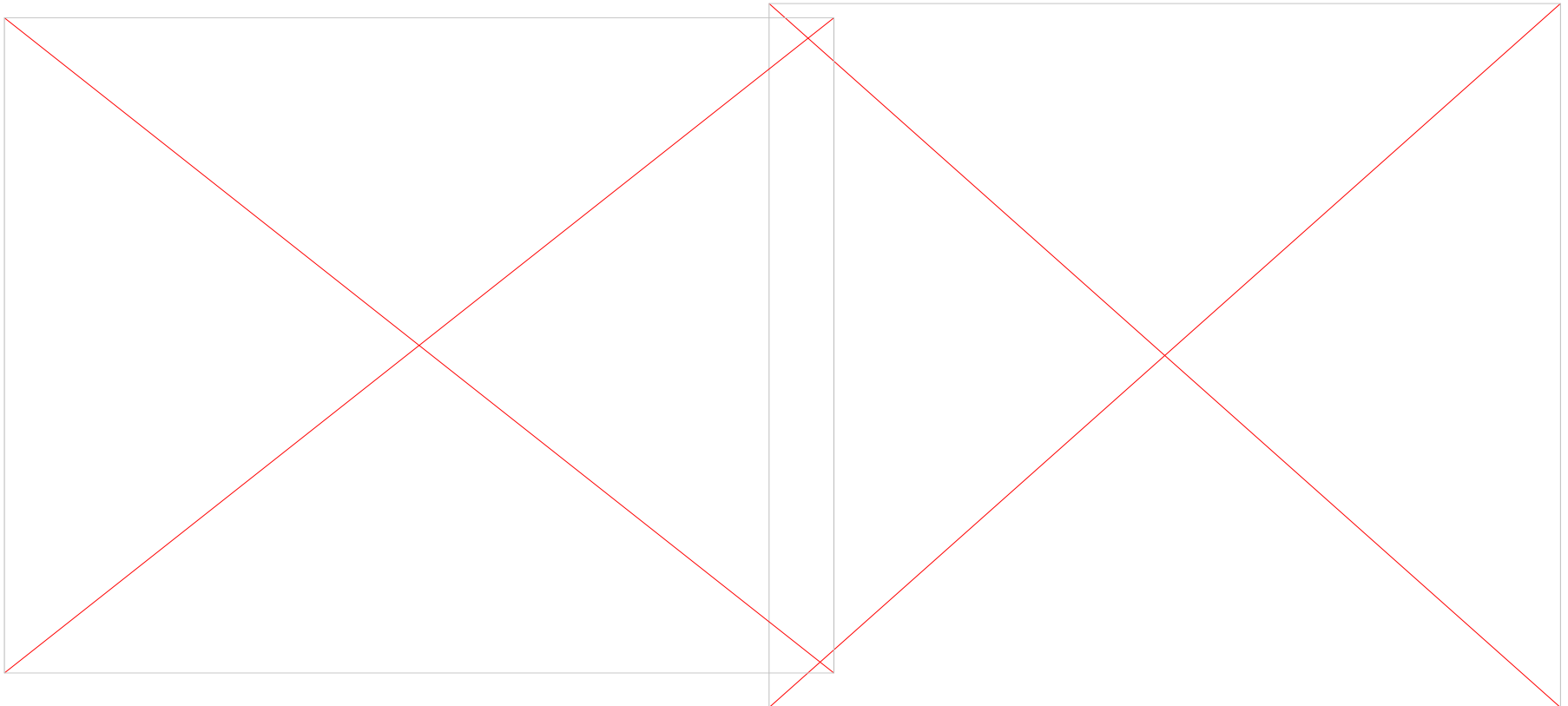


Project Operation Example

- Example: eliminate the *dept_name* attribute of *instructor*
- Query:

$\Pi_{ID, name, salary}(instructor)$

Result (order is irrelevant)





Project Operation

- A unary operation that returns its argument relation, with certain attributes left out.
- Notation:

$$\Pi_{A_1, A_2, A_3 \dots A_k}(r)$$

where $A_1, A_2, \dots, A_k$ are attribute names and r is a relation name.

- The result is defined as the relation of k columns obtained by erasing the columns that are not listed
- **Duplicate rows removed from result**, since relations are sets



Composition of Relational Operations

- The result of a relational-algebra operation is relation and therefore of relational-algebra operations can be composed together into a **relational-algebra expression**.
- Consider the query -- Find the names of all instructors in the Physics department.

$$\Pi_{name}(\sigma_{dept_name = "Physics"}(instructor))$$

- Instead of giving the name of a relation as the argument of the projection operation, we give an expression that evaluates to a relation.



Cartesian-Product Operation

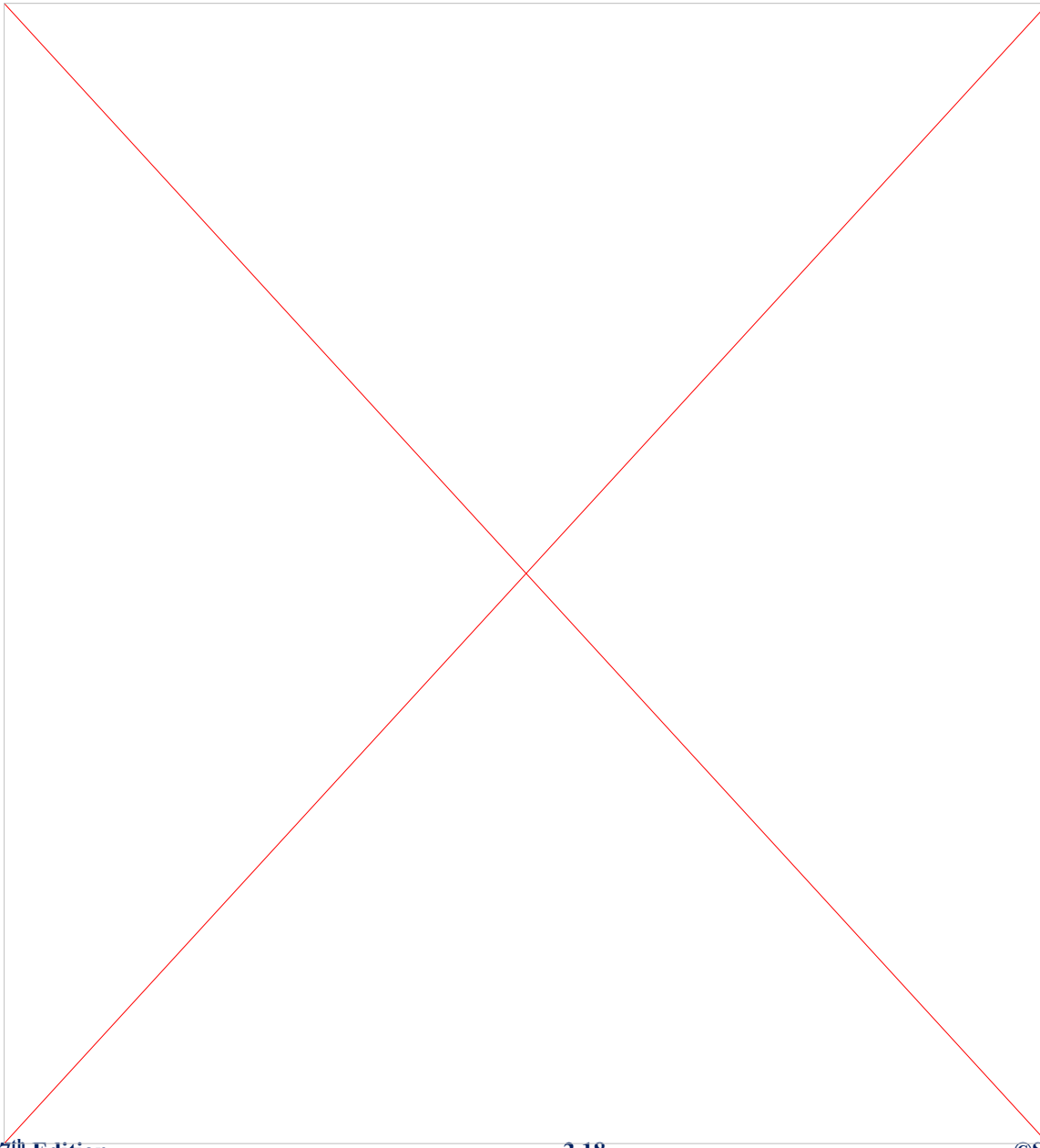
- The Cartesian-product operation (denoted by $\times$) allows us to combine information from any two relations.
- Example: the Cartesian product of the relations *instructor* and *teaches* is written as:

instructor $\times$ *teaches*

- We construct a tuple of the result out of each possible pair of tuples: one from the *instructor* relation and one from the *teaches* relation (see next slide)
- Since the instructor *ID* appears in both relations we distinguish between these attribute by attaching to the attribute the name of the relation from which the attribute originally came.
 - *instructor.ID*
 - *teaches.ID*



The *instructor X teaches table*





Join Operation

- The Cartesian-Product

instructor X teaches

associates every tuple of *instructor* with every tuple of *teaches*.

- Most of the resulting rows have information about instructors who did NOT teach a particular course.
- To get only those tuples of “*instructor X teaches*” that pertain to instructors and the courses that they taught, we write:

$\sigma_{instructor.id = teaches.id} (instructor \times teaches)$

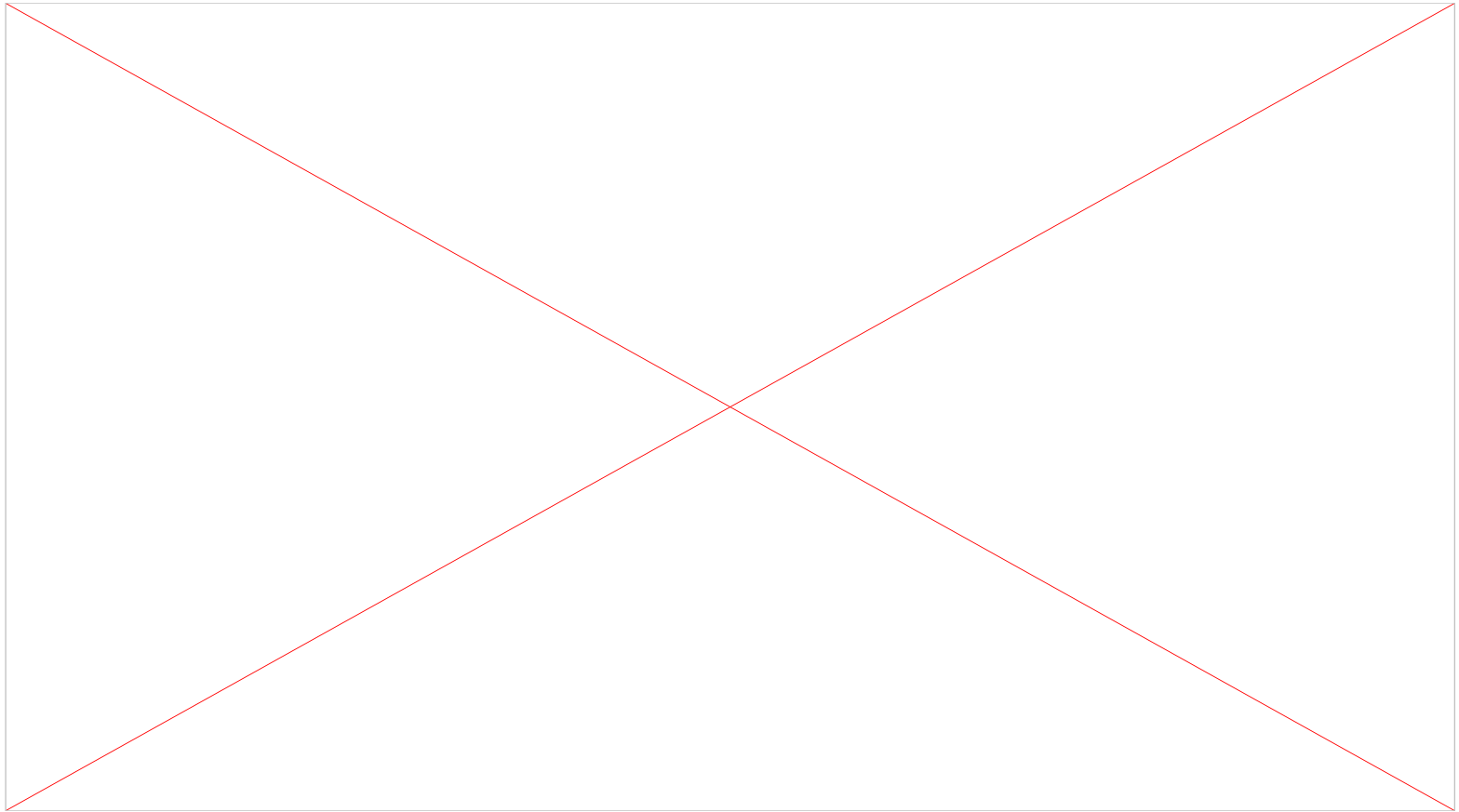
- We get only those tuples of “*instructor X teaches*” that pertain to instructors and the courses that they taught.
- The result of this expression, shown in the next slide



Join Operation (Cont.)

- The table corresponding to:

$$\sigma_{instructor.id = teaches.id}(instructor \times teaches))$$





Join Operation (Cont.)

- The **join** operation allows us to combine a select operation and a Cartesian-Product operation into a single operation.
- Consider relations $r(R)$ and $s(S)$
- Let “theta” be a predicate on attributes in the schema R “union” S . The join operation $r \bowtie_{\theta} s$ is defined as follows:

$$r \bowtie_{\theta} s = \sigma_{\theta} (r \times s)$$

- Thus

$$\sigma_{instructor.id = teaches.id} (instructor \times teaches)$$

- Can equivalently be written as

$$instructor \bowtie_{instructor.id = teaches.id} teaches.$$



Union Operation

- The union operation allows us to combine two relations
- Notation: $r \cup s$
- For $r \cup s$ to be valid.
 1. r, s must have the *same* **arity** (same number of attributes)
 2. The attribute domains must be **compatible** (example: 2nd column of r deals with the same type of values as does the 2nd column of s)
- Example: to find all courses taught in the Fall 2017 semester, or in the Spring 2018 semester, or in both

$$\Pi_{course_id} (\sigma_{semester="Fall" \wedge year=2017}(section)) \cup$$
$$\Pi_{course_id} (\sigma_{semester="Spring" \wedge year=2018}(section))$$



section table

<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>building</i>	<i>room_number</i>	<i>time_slot_id</i>
BIO-101	1	Summer	2017	Painter	514	B
BIO-301	1	Summer	2018	Painter	514	A
CS-101	1	Fall	2017	Packard	101	H
CS-101	1	Spring	2018	Packard	101	F
CS-190	1	Spring	2017	Taylor	3128	E
CS-190	2	Spring	2017	Taylor	3128	A
CS-315	1	Spring	2018	Watson	120	D
CS-319	1	Spring	2018	Watson	100	B
CS-319	2	Spring	2018	Taylor	3128	C
CS-347	1	Fall	2017	Taylor	3128	A
EE-181	1	Spring	2017	Taylor	3128	C
FIN-201	1	Spring	2018	Packard	101	B
HIS-351	1	Spring	2018	Painter	514	C
MU-199	1	Spring	2018	Packard	101	D
PHY-101	1	Fall	2017	Watson	100	A



Union Operation (Cont.)

- Result of:

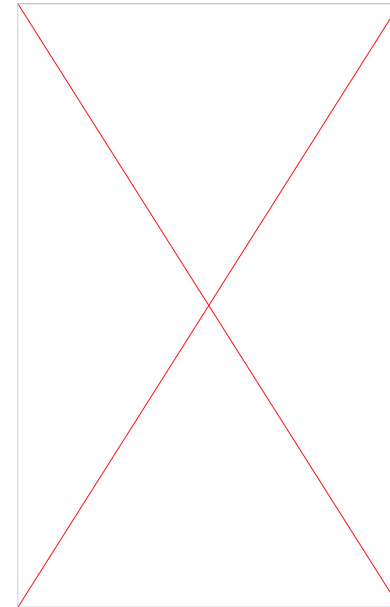
$$\Pi_{course_id} (\sigma_{semester="Fall" \wedge year=2017}(section)) \cup \Pi_{course_id} (\sigma_{semester="Spring" \wedge year=2018}(section))$$

<i>course_id</i>
CS-101
CS-347
PHY-101

U

<i>course_id</i>
CS-101
CS-315
CS-319
FIN-201
HIS-351
MU-199

=



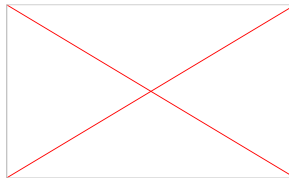


Set-Intersection Operation

- The set-intersection operation allows us to find tuples that are in both the input relations.
- Notation: $r \cap s$
- Assume:
 - r, s have the *same arity*
 - attributes of r and s are compatible
- Example: Find the set of all courses taught in both the Fall 2017 and the Spring 2018 semesters.

$$\Pi_{course_id} (\sigma_{semester="Fall" \wedge year=2017}(section)) \cap \Pi_{course_id} (\sigma_{semester="Spring" \wedge year=2018}(section))$$

- Result

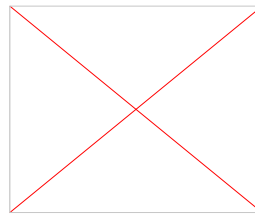




Set Difference Operation

- The set-difference operation allows us to find tuples that are in one relation but are not in another.
- Notation $r - s$
- Set differences must be taken between **compatible** relations.
 - r and s must have the **same** arity
 - attribute domains of r and s must be compatible
- Example: to find all courses taught in the Fall 2017 semester, but not in the Spring 2018 semester

$$\Pi_{course_id}(\sigma_{semester="Fall" \wedge year=2017}(section)) - \Pi_{course_id}(\sigma_{semester="Spring" \wedge year=2018}(section))$$





The Assignment Operation

- It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables.
- The assignment operation is denoted by $\leftarrow$ and works like assignment in a programming language.
- Example: Find all instructor in the “Physics” and Music department.

$$Physics \leftarrow \sigma_{dept_name = \text{“Physics”}}(instructor)$$
$$Music \leftarrow \sigma_{dept_name = \text{“Music”}}(instructor)$$
$$Physics \cup Music$$

- With the assignment operation, a query can be written as a sequential program consisting of a series of assignments followed by an expression whose value is displayed as the result of the query.



The Rename Operation

- The results of relational-algebra expressions do not have a name that we can use to refer to them. The rename operator, ρ , is provided for that purpose
- The expression:

$$\rho_x(E)$$

returns the result of expression E under the name x

- Another form of the rename operation:

$$\rho_{x(A1, A2, \dots, An)}(E)$$



Aggregate Functions

- These functions operate on the multiset (set with duplicates) of values of a column of a relation, and return a value
 - avg:** average value
 - min:** minimum value
 - max:** maximum value
 - sum:** sum of values
 - count:** number of values



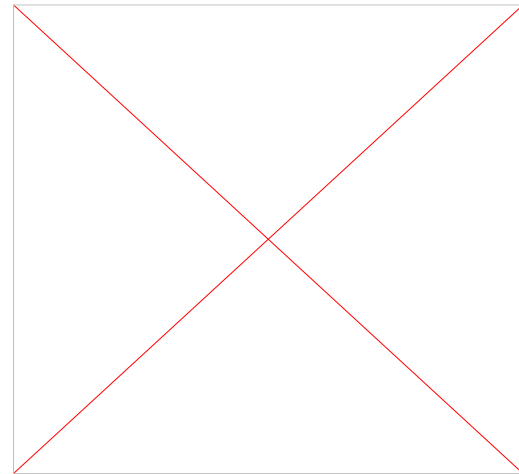
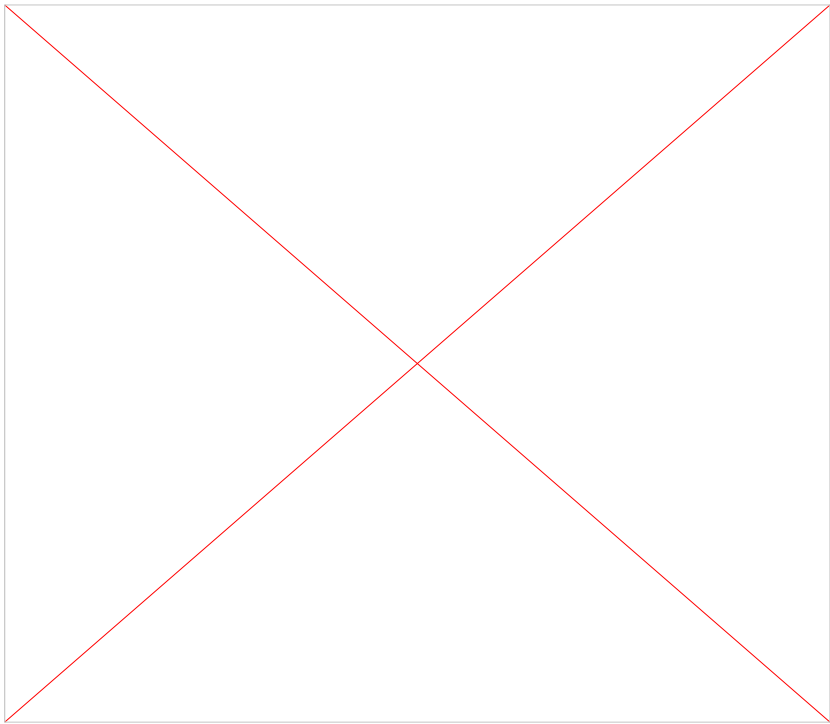
Aggregate Functions Examples

- Find the average salary of instructors
 - $\gamma_{avg(salary)}(instructor)$
- Find the total number of courses in 2018
 - $\gamma_{count(course_id)}(\sigma_{year=2018}(section))$



Aggregate Functions – Group By

- Find the average salary of instructors in each department
 - $dept_name \gamma_{avg(salary)}(instructor)$





Equivalent Queries

- There is more than one way to write a query in relational algebra.
- Example: Find information about courses taught by instructors in the Physics department with salary greater than 90,000

- Query 1

$$\sigma_{dept_name="Physics"} \wedge salary > 90,000 (instructor)$$

- Query 2

$$\sigma_{dept_name="Physics"} (\sigma_{salary > 90,000} (instructor))$$

- The two queries are not identical; they are, however, equivalent -- they give the same result on any database.



Equivalent Queries

- There is more than one way to write a query in relational algebra.
- Example: Find information about courses taught by instructors in the Physics department
- Query 1

$\sigma_{dept_name = "Physics"}(instructor \bowtie_{instructor.ID = teaches.ID} teaches)$

- Query 2

$(\sigma_{dept_name = "Physics"}(instructor)) \bowtie_{instructor.ID = teaches.ID} teaches$

- The two queries are not identical; they are, however, equivalent -- they give the same result on any database.



Pitfalls in relational database design

- Data Redundancy and Operational Anomalies
 - Insertion Anomaly
 - Deletion Anomaly
 - Update Anomaly
- Poor Normalization Choices
 - Under-Normalization (Data Redundancy)
 - Over-Normalization
- Misusing or Ignoring Indexes
 - Missing Essential Indexes
 - Over-Indexing
- Weak Referential Integrity
 - Skipping Foreign Keys
 - Using God Tables



Pitfalls in relational database design

- Poor Primary Key Selection
 - Using Volatile Natural Keys
 - Using Inappropriate Data Types
- Lacking Data Constraints
 - Allowing Too Many NULLs
 - Ignoring CHECK Constraints
- Designing for the Present, Not the Future
 - Hardcoded Column Enums
 - Ignoring Scalability



Thank You